Smartphone-Based Maize Leaf Disease Detection Using YOLO26

Himanshu Singh, Jayesh Choudhary, Pitamber Jagawat, Ajay Pratap, *Senior Member, IEEE*

Abstract—Foliar diseases such as northern leaf blight, gray leaf spot, and common rust cause substantial yield losses in maize crop. The traditional field diagnosis techniques rely largely on visual inspection of leaf symptoms. This approach is labor-intensive and requires expert knowledge to distinguish diseases with similar visual symptoms. To handle that, convolutional neural networks can achieve high accuracy in image-based plant disease recognition. However, their computational and memory requirements limit their deployment on low-end processing devices. Nano-scale YOLO classifiers enable on-device inference with lower processing resource consumption. However, their classification head uses global average pooling followed by a single linear layer, which may not preserve the lesion-level features needed to distinguish visually similar disease classes. To address this limitation, this paper proposes the Dual-Stream Attention Classifier (DSAC), a lightweight classification head for a frozen, ImageNet-pretrained YOLO26n-cls backbone. DSAC applies channel and spatial attention to emphasize lesion regions in maize leaf images. It then combines intermediate backbone features through a gated Multi-Scale Feature Pyramid (MSFP). The features are then combined using average and max pooling to capture different aspects of the feature representation. These pooled features are then passed to a residual multilayer perceptron classifier. A label-smoothed focal loss is utilized to address class imbalance and reduce overconfident predictions between visually similar diseases such as blight and gray leaf spot. The proposed method is evaluated on a six-class maize leaf dataset containing 5,906 image samples. DSAC achieves a Top-1 accuracy of 98.09% and a macro-averaged precision–recall AUC of 0.9948 outperforming baseline methods. DSAC also improves fall armyworm recall from 0.79 to 0.94 compared with the YOLO26n-cls baseline while maintaining CPU inference latency below 10 ms. The model is deployed offline in an Android application using ONNX Runtime and achieves an end-to-end latency of 12–15 ms on a Google Pixel 7 Pro. Evaluation results show that DSAC provides accurate maize disease classification while maintaining the computational efficiency required for mobile deployment.

Index Terms—smartphone-based detection, maize disease classification, YOLO26n-cls, on-device inference, mobile deployment, precision agriculture, deep learning

1 Introduction

Maize is one of the most widely cultivated cereal crops and a major source of food, animal feed, and industrial raw material. However, maize productivity is significantly affected by foliar diseases and pests, such as northern leaf blight, gray leaf spot, common rust, and fall armyworm infestation. Early and accurate detection enables timely intervention, prevents disease spread, and reduces unnecessary use of pesticides. However, a large section of farmers still rely on visual inspection, which is labor-intensive, requires expert knowledge of disease symptoms, and is difficult to scale across large farmlands [1]. To handle that, deep learning based techniques can be utilized that has substantially advanced image-based plant disease recognition.

Convolutional neural networks (CNNs), such as ResNet and EfficientNet learn disease features directly from leaf images. These models achieve high detection accuracy across multiple crops and disease types. However, this accuracy comes at a considerable computational and memory cost, which limits their deployment on the low-power devices typically available in the field. A traditional approach to handle this is to offload inference to cloud servers. However, cloud-based techniques require reliable internet connectivity which is often missing in rural farmland regions. Thus, there is need of a lightweight processing model that can effectively balance accuracy and processing resource consumption (CPU, memory) while providing on-device inference.

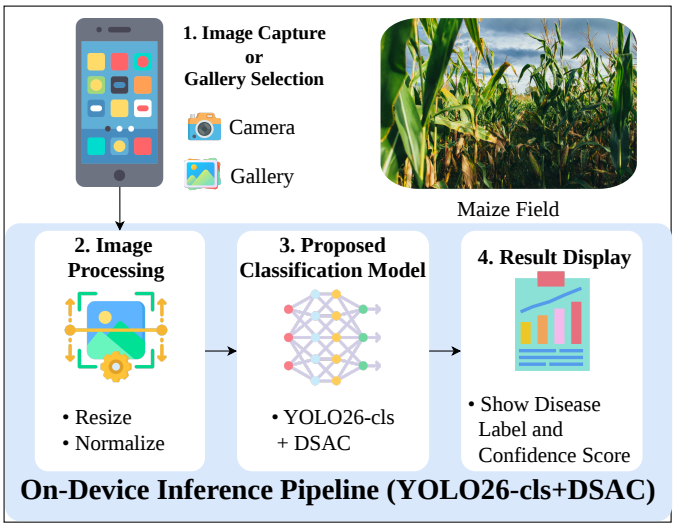


Fig. 1: Proposed disease detection workflow.

Achieving high accuracy with lightweight model is difficult for maize disease detection. First, images exhibit

• *Himanshu Singh, Jayesh Choudhary, Pitamber Jagawat and Ajay Pratap are with the Department of Computer Science and Engineering, Indian Institute of Technology (Banaras Hindu University), Varanasi 221005, India. E-mail: {himanshusingh.rs.cse21, jayesh.choudhary.cse23, pitamber.jagawat.cse23 ajay.cse}@iitbhu.ac.in.*

large variations in illumination, viewpoint, and background clutter, including soil, weeds, and neighboring leaves. The model may learn from these background elements instead of the diseased leaf regions. Further, several maize diseases are visually similar making model difficult to distinguish. For example, blight and gray leaf spot both produce long, narrow lesions with similar color and texture at certain growth stages, so correct classification depends on fine, localized lesion details. Further, dataset is often imbalanced with some classes represented by considerably fewer samples than healthy leaves or common diseases. Under these circumstances, a wrong prediction with high confidence may lead the farmer to apply an incorrect treatment instead of consulting an agricultural expert.

Existing approaches improve either accuracy or model efficiency but not always both. Maize-specific CNNs are compact but either fail to distinguish visually similar diseases or cannot focus on lesion regions. Attention-based models, such as VGG16 with squeeze-and-excitation blocks, distinguish diseases more accurately but are too large for low-end processing devices. Multimodal approaches require sensor data in addition to clear and bright images. Nano-sized YOLO classifiers, including YOLOv8n-cls, YOLO11n-cls, and the recent YOLO26n-cls balance accuracy and speed well and can be readily converted for low-end device deployment. However, these models use a simple classification head that averages the final feature map and passes the result to a single linear layer. For fine-grained disease recognition, YOLO nano-scale classifiers have three limitations. First, equal weighting of lesion and background regions limits lesion-focused representation. Second, intermediate-layer lesion details are discarded. Third, global average pooling weakens responses from small or scattered lesions. These limitations indicate that improving the classification head can enhance fine-grained disease recognition without increasing the backbone size.

Based on above-mentioned limitations, this paper proposes a lightweight maize leaf disease classification framework for resource-constrained devices. The framework uses an ImageNet-pretrained YOLO26n-cls backbone with a fixed feature extractor and a Dual-Stream Attention Classifier (DSAC). DSAC applies the Convolutional Block Attention Module (CBAM) to emphasize disease-affected regions. A gated Multi-Scale Feature Pyramid (MSFP) combines features from three intermediate backbone layers to retain lesion details. Average and max pooling retain global leaf features and strong activations from localized lesion regions. A residual multilayer perceptron performs the final classification. The model uses label-smoothed focal loss to limit overconfident predictions for visually similar diseases and emphasize poorly classified samples. The YOLO26n-cls backbone remains unchanged during training, and only the DSAC head is optimized. The trained model is exported to ONNX and INT8-quantized CoreML formats for efficient edge inference on a smartphone.

The main contributions of this paper are summarized as follows:

- We propose DSAC as a lightweight classification head for YOLO26n-cls. DSAC combines channel–spatial attention, gated multi-scale feature fusion, dual pooling, and a residual classifier with a fixed pretrained backbone.
- We utilize label-smoothed focal loss with stratified data partitioning and MixUp augmentation to address class imbalance and overconfidence between visually similar diseases.
- We evaluate the proposed model against eight baseline models. DSAC achieves 98.09% Top-1 accuracy and a macro-averaged precision–recall AUC of 0.9948, with a 2.14% improvement over YOLO26n-cls. DSAC also achieves the highest or equal-highest F1-score across all six classes and improves Fall Armyworm recall to 0.94. CPU inference latency remains below 10 ms.
- We develop a prototype application for on-device maize disease recognition and evaluate the proposed model on mobile hardware. The Android prototype using Flutter and ONNX runtime achieves 12–15 ms end-to-end latency on a Google Pixel 7 Pro. The INT8 CoreML model achieves 2.4 ms inference latency on Apple hardware, with less than 0.5 percentage-point accuracy loss after export and quantization.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the proposed DSAC methodology. Section 4 presents the experimental results and discussion. Section 5 concludes the paper.

2 Related Works

Deep learning has greatly improved plant disease detection. Manual inspection is slow, labor-intensive, and often makes mistakes at scale. Early models like ResNet achieve strong classification results. However, they require too much computing power for smartphones, Raspberry Pi units, and other small edge devices.

Researchers then created lighter models specifically for maize. Models like MaizeNet [2] and Corn-CNN [3] are small. However, they miss fine details and struggle to separate visually similar diseases. LW-CNN [4] runs well on small hardware, but it lacks attention mechanisms. E2E Fusion [5] needs extra sensor data, which a standard phone camera cannot provide. VGG16-SE [6] uses attention to separate diseases better. But it is simply too large for mobile deployment. LightET-FusionNet [7] uses a lightweight ensemble approach. It scales well, but ensemble models are usually too slow for real-time phone use.

Very recent papers from 2026 try to fix these issues. LWMSCNN-SE [8] uses multi-scale features and SE attention. It is very fast on edge devices. However, it uses a standard pooling head. This loses the small lesion details needed to separate similar diseases. Srinivasan and Kokil [9] proposed a residual attention framework. It improves accuracy, but it fine-tunes the entire network backbone. This takes much more time and memory during training and inference.

Nano YOLO models like YOLOv8n-cls, YOLO11n-cls, [10] and YOLO26n-cls [11] are also very fast. They run easily on phones. But they use a very simple classification head. They just average the final features. This makes it hard to tell visually similar diseases apart.

TABLE 1: Comparison with Related Work

Model	Attention Mechanism	Multi-Scale Fusion	Dual Pooling	Residual Classifier Head	On-Device Deployment	Sub-15ms CPU Latency	Remarks
MaizeNet [2]	✓	✓	×	×	✓	✓	Maize-specific but generalises poorly between visually similar classes.
Corn-CNN [3]	×	×	×	×	✓	×	Very few parameters but accuracy ceiling is modest.
LW-CNN [4]	×	×	×	×	✓	×	Runs on embedded hardware but has no attention or multi-scale features.
E2E Fusion [5]	×	×	×	×	×	×	Needs extra sensor input with RGB – not usable with single phone camera.
VGG16-SE [6]	✓	×	×	×	×	×	SE attention helps discrimination, but too large for mobile deployment.
LightET-FusionNet [7]	×	✓	×	✓	✓	×	Lightweight ensemble model, but fusion overhead prevents sub-10ms latency.
LWMSCNN-SE [8]	✓	✓	×	×	✓	✓	Highly efficient multi-scale + SE attention, but lacks dual-pooling for fine details.
Residual Attention [9]	✓	×	×	✓	✓	×	Good accuracy, but fine-tunes the entire backbone, increasing memory and training cost.
YOLOv8n-cls, YOLO11n-cls [10]	×	×	×	×	✓	✓	Nano detection-derived classifiers, the closest direct baselines here.
YOLO26n-cls [11]	×	×	×	×	✓	✓	Backbone used to build our proposed DSAC head.
Proposed (DSAC)	✓	✓	✓	✓	✓	✓	Attention + multi-scale fusion + dual pooling + residual head for accurate, low-latency on-device inference.

TABLE 2: List of Symbols

Symbol	Description
$\mathcal{L}_{\mathbf{cls}}$	Loss function
ε	Label-smoothing coefficient
γ	Focal-loss parameter
$\mathcal{D}$	Labelled training dataset
θ	Model parameters
p_m	MixUp interpolation coefficient
g_i	Learned scalar gate for MSFP fusion stream i
β_i	Logit of gate g_i ($g_i = \sigma(\beta_i)$)
F	Backbone feature map $\in \mathbb{R}^{B\times C\times H\times W}$
$\mathbf{z}$	Dual-pooled feature vector $\in \mathbb{R}^{B\times 2C}$
r	Channel-attention bottleneck ratio
B	Batch Size
H_{res}	Resized image height
W_{res}	Resized image width
H	Feature Map Height
W	Feature Map Width
C	Number of Channels
mAP	Macro-averaged precision–recall AUC
h_1	First intermediate activation in residual classifier head
h_2	Second intermediate activation in residual classifier head
W_1	Weight matrix of first linear layer
W_2	Weight matrix of second linear layer
W_3	Weight matrix of output linear layer
W_s	Skip projection matrix

Our model fixes these specific problems. We use a YOLO26n-cls backbone and keep it frozen to save memory. We add a new head called DSAC. It uses CBAM attention, mixes multi-scale features, and uses dual pooling. It also has a residual classifier. This gives high accuracy and stays fast enough for real-time phone use. Table 1 summarizes these architectures.

3 METHODOLOGY

The proposed method classifies maize leaf disease from images using a YOLO26n-cls backbone [11] followed by the Dual-Stream Attention Classifier (DSAC), and the trained model is quantized for offline mobile deployment. Each input image is resized to $H_{res} \times W_{res}$ and normalized, 384×384 was chosen to keep lesion and shape details needed to separate visually similar diseases such as Gray Leaf Spot and Blight. The backbone supplies feature maps from three intermediate layers ($L2$, $L6$, and $L8$), where shallower layers retain local detail and deeper layers capture broader disease patterns. The final backbone feature map is first refined by CBAM to emphasize diseased regions and suppress the background. In the Multi-Scale Feature Pyramid (MSFP), each intermediate map is projected with a 1×1 convolution, scaled by a learned gate, and added to the refined map, which combines local lesion detail with deeper disease patterns. The fused output then passes through the Dual Pooling Aggregation (DPA) block, which forms a compact representation for the Residual Classifier Head to predict the disease class.

The architecture of the proposed DSAC is described in the following subsection.

3.1 DSAC Architecture Overview

Conventional classification heads aggregate only the final backbone features before predicting the class. This may not preserve the visual information needed when disease identification depends on lesion location, texture, shape, and distribution across the leaf surface, and it ignores lesion details captured at earlier backbone layers. To address this,

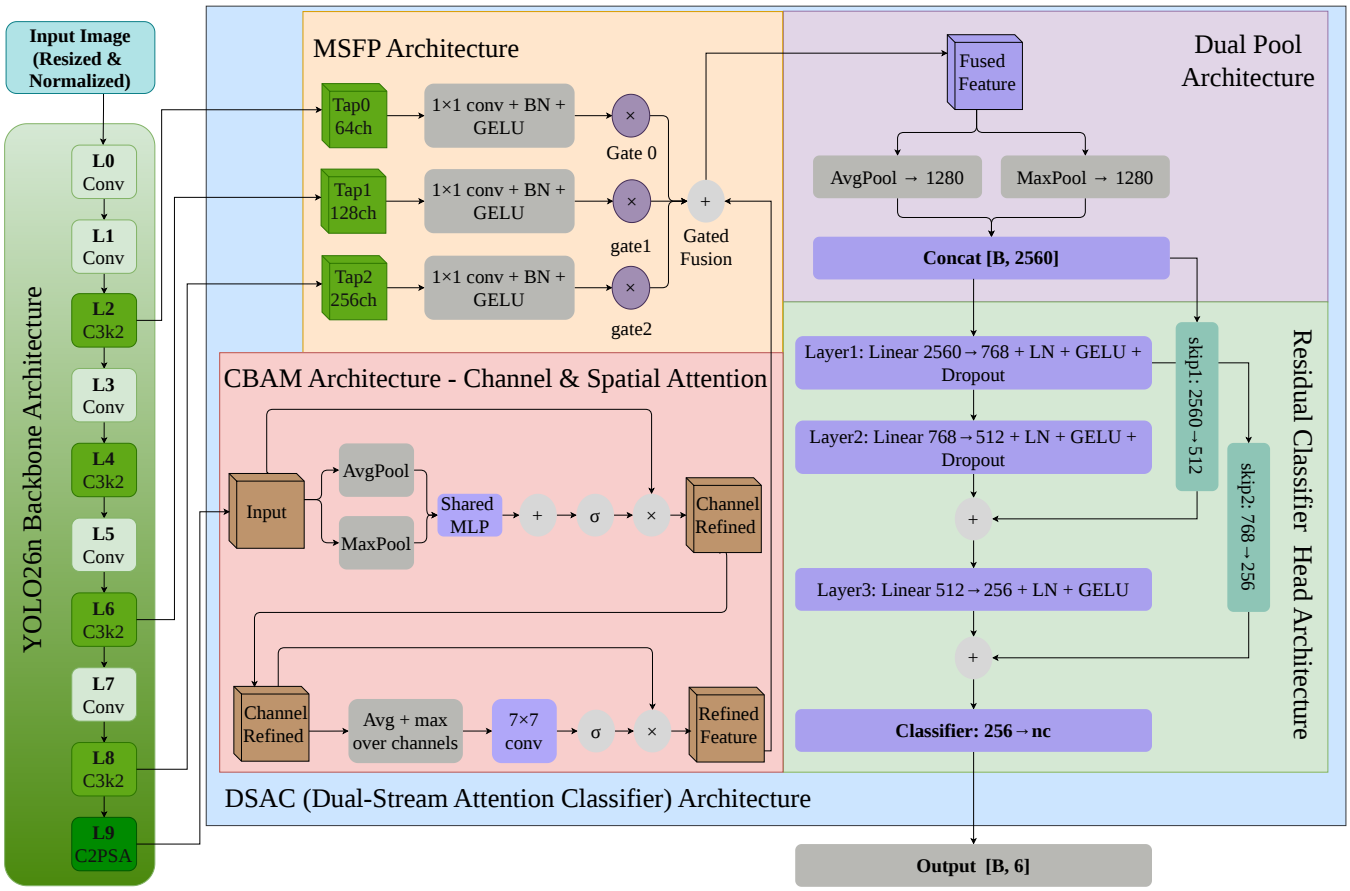


Fig. 2: Layer-by-layer architecture of DSAC(proposed model)

the proposed DSAC is placed after the YOLO26n-cls backbone and combines features from different backbone depths with the final feature representation through four modules that progressively refine and aggregate information from disease affected regions of the leaf. Fig. 2 illustrates the architecture. CBAM addresses spatial irrelevance and noise in the captured leaf image, MSFP captures multi-scale disease patterns, the Dual Pool module preserves complementary feature information for accurate recognition, the Residual Head facilitates gradient flow for effective deep feature learning, and the label smoothing with focal loss module reduces overconfidence and improves discrimination among visually similar diseases. Together, these modules capture lesion information from different feature levels, emphasize relevant leaf regions, aggregate complementary image information, and perform the final disease classification, as formalized below.

$$\begin{aligned}
\mathbf{F}' &= \mathrm{CBAM}(\mathbf{F}_{\mathrm{backbone}}) \\
\mathbf{F}'' &= \mathrm{MSFP}(\mathbf{F}', \mathbf{F}_{\mathrm{scales}}) \\
\mathbf{z} &= \mathrm{DualPool}(\mathbf{F}'') \\
\hat{\mathbf{y}} &= \mathrm{ResHead}(\mathbf{z}) \\
\mathcal{L} &= \mathcal{L}_{\mathrm{cls}}\left(\hat{\mathbf{y}}, \mathbf{y}\right)
\end{aligned}$$

Fig. 2 illustrates the DSAC architecture for maize leaf disease classification. DSAC consists of five components as discussed in the following subsections.

3.2 Convolutional Block Attention Module

CBAM is applied to the final feature map extracted by the YOLO26n-cls backbone before subsequent feature processing. A classification backbone is optimized to distinguish disease classes, but its final feature representation may also contain information from unaffected leaf regions and the image background. This can be less suitable when disease identification depends on the location and shape of lesions on the maize leaf. CBAM addresses this issue by refining the feature map $\mathbf{F}$, the 1280-channel feature map from the last Conv layer of YOLO26n-cls before pooling, through channel and spatial attention [12]. The channel attention identifies informative feature channels, while the spatial attention emphasizes relevant regions within the maize leaf image. These two operations are expressed as [12]:

$$\mathbf{F}_c = \mathbf{F} \odot \sigma\left(\mathrm{MLP}(\mathrm{AvgPool}(\mathbf{F})) + \mathrm{MLP}(\mathrm{MaxPool}(\mathbf{F}))\right), \tag{1}$$

$$\mathbf{F}' = \mathbf{F}_c \odot \sigma\left(f^{7\times7}\left([\mathrm{AvgPool}_c(\mathbf{F}_c); \mathrm{MaxPool}_c(\mathbf{F}_c)]\right)\right) \tag{2}$$

The channel attention in Eq. (1) refines the backbone features by assigning greater importance to channels associated with disease-related leaf patterns. The spatial attention in Eq. (2) subsequently emphasizes regions containing disease symptoms [12].

3.3 Multi-Scale Feature Pyramid

The MSFP is introduced to incorporate feature information from different stages of the backbone into the final feature representation. As the maize leaf image passes through the backbone, the intermediate layers retain lesion details that may not be fully represented in the final feature map. MSFP therefore uses feature maps extracted at approximately 20%, 60%, and 80% of the backbone depth and combines them with the CBAM-refined feature map [13]. This operation is defined as:

$$\mathbf{F}'' = \mathbf{F}' + \sum_{i=1}^{3} g_i \cdot \mathrm{Proj}_i\left(\mathbf{F}^{(i)}\right), \tag{3}$$

where $\mathbf{F}^{(i)}$ denotes the feature map extracted at the (i)-th backbone depth and $\mathrm{Proj}_i(\cdot)$ represents the 1×1 projection. The projected feature maps are spatially aligned using adaptive average pooling before fusion. The gate $g_i = \sigma(\beta_i)$ controls the contribution of each feature map. The fused representation combines lesion details from the intermediate layers with the disease-related information retained in the final feature map.

3.4 Dual-Pooling Aggregation

The feature map obtained after MSFP contains information related to both the overall appearance of the maize leaf and localized disease symptoms. Average pooling and max pooling capture these characteristics differently. Average pooling represents the overall feature distribution, whereas max pooling retains the strongest feature responses associated with localized lesion patterns. Therefore, both pooling operations are used to construct the input to the classifier as given below [12]:

$$\mathbf{z} = [\mathrm{AvgPool}(\mathbf{F}'') \parallel \mathrm{MaxPool}(\mathbf{F}'')] \in \mathbb{R}^{B\times 2C}, \tag{4}$$

where $\mathbf{z}$ represents the aggregated feature vector. The concatenated representation preserves information from the complete maize leaf feature map together with strong localized responses from disease-affected regions. This representation is subsequently provided to the Residual Classifier Head for maize leaf disease classification.

3.5 Deep Residual Classifier Head

The pooled feature vector $\mathbf{z} \in \mathbb{R}^{B\times 2C}$ (where $2C = 2560$) contains information from the overall maize leaf appearance and localized disease symptoms. The Residual Classifier Head transforms these features into the disease class scores through three successive fully connected layers. Two residual skip connections are included to preserve gradient flow and prevent information loss during this deep transformation [14], [15]. The classifier is defined as:

$$\mathbf{h}_1 = \mathrm{Drop}_{0.3}\left(\mathrm{GELU}\left(\mathrm{LN}(W_1\mathbf{z})\right)\right), \quad W_1 \in \mathbb{R}^{512\times 2C}, \tag{5}$$

$$\mathbf{h}_2 = \mathrm{Drop}_{0.3}\left(\mathrm{GELU}\left(\mathrm{LN}(W_2\mathbf{h}_1)\right)\right) + W_{s1}\mathbf{z}, \quad W_2 \in \mathbb{R}^{256\times 512}, \tag{6}$$

$$\mathbf{h}_3 = \mathrm{GELU}\left(\mathrm{LN}(W_3\mathbf{h}_2)\right) + W_{s2}\mathbf{h}_1, \quad W_3 \in \mathbb{R}^{nc\times 256}, \tag{7}$$

$$\hat{\mathbf{y}} = W_4\mathbf{h}_3, \quad W_4 \in \mathbb{R}^{256\times 2C}, \tag{8}$$

where $W_{s1} \in \mathbb{R}^{2C\times 512}$ projects the initial pooled feature vector to match the dimension of $\mathbf{h}_2$, and $W_{s2} \in \mathbb{R}^{768\times 256}$ projects the output of the first hidden layer to match the dimension of $\mathbf{h}_3$. The residual connections combine the original representations with the transformed features before the subsequent layers. The resulting output $\hat{\mathbf{y}}$ contains the scores for the n_c maize leaf disease classes.

3.6 Label Smoothing and Focal-Aware Training

Blight and Gray Leaf Spot are visually close enough that a model trained on hard, one-hot labels can become confidently wrong about the boundary between them, which is a worse outcome in practice than being uncertain as a confident misclassification gives a farmer no signal that the prediction deserves a second look.

Training addresses this at the loss level instead of changing the architecture. The model is trained with a label-smoothed, focal-weighted cross-entropy loss (9) [16]–[18]. Label smoothing (ε) replaces the one hot target y_k with the softened target q_k, which keeps the model from being pushed toward absolute certainty and is used to reduce overconfidence between visually similar classes such as Blight and Gray Leaf Spot. The focal factor $(1-p_k)^\gamma$ down-weights classes the model already predicts confidently and up-weights harder, boundary-adjacent examples [18]. When MixUp is applied, y_k is the mixed label.

$$L_{\mathrm{cls}} = -\sum_{k=1}^{K} q_k(1-p_k)^\gamma \log p_k, \tag{9}$$

where

$$q_k = (1-\varepsilon)y_k + \frac{\varepsilon}{K}, \qquad p = \mathrm{softmax}(\hat{y}).$$

Fig. 3 reflects the effect of this combined objective on training behavior. The loss curve descends smoothly and the gap between training and validation loss stays small throughout, consistent with a model that is not simply memorizing the majority classes at the expense of harder boundary cases.

3.7 DSAC Training Pipeline

Initially, labeled dataset $\mathcal{D}$ is divided into training, validation, and test sets using a 70/15/15% split (line 1, Algorithm 1). Next, each image is resized to $H_{res} \times W_{res}$ pixels and normalized using the ImageNet channel statistics (line 2). The ImageNet-pretrained YOLO26n-cls backbone is loaded with its parameters θ_b fixed (line 3). During training, the batch Normalization layers of YOLO26n-cls are kept in evaluation mode to prevent updates to their running statistics. This prevents unreliable updates to batch normalization statistics caused by small or heterogeneous batches. Feature maps from layers L2, L6, and L8 are obtained during the forward pass for subsequent processing. These feature maps are represented by $\mathbf{F}^{(i)}{}_{i=1}^{3}$. Subsequently, the YOLO26n-cls head is replaced by DSAC and its parameters θ_h are optimized during training (lines 4–5). The optimization is performed using AdamW with the learning rate increased over first five epochs and then decayed according to a cosine schedule. Moreover, the best validation accuracy A_{best} is

Algorithm 1 DSAC Training Pipeline

Input: Labeled dataset $\mathcal{D} = (\mathbf{x}_i, y_i)_{i=1}^{N}$ with $K = 6$ classes; number of epochs E
Output: Trained model M^*
Step 1: Data splitting
1: Split $\mathcal{D}$ into $\mathcal{D}_{tr}$, $\mathcal{D}_{val}$, and $\mathcal{D}_{te}$ using a 70%/15%/ 15% stratified split
Step 2: Preprocessing
2: Resize images to $H_{res} \times W_{res}$ and normalize using ImageNet statistics
Step 3: Model construction
3: Load ImageNet-pretrained YOLO26n-cls and fix its backbone parameters θ_b
4: Set the backbone to evaluation mode to prevent updates to Batch Normalization statistics
5: Extract $\mathbf{F}^{(i)}{}_{i=1}^{3}$ from layers L2, L6, and L8 during the forward pass
6: Replace the YOLO26n-cls classification head with DSAC and define its parameters as θ_h
Step 4: Training
7: Initialize AdamW with $\eta_0 = 5 \times 10^{-4}$ and weight decay 5×10^{-4}; set $A_{\text{best}} \leftarrow 0$
8: **for** $e = 1, \ldots, E$ **do**
9: **for** each mini-batch $(\mathbf{X}, \mathbf{Y}) \subset \mathcal{D}_{tr}$ **do**
10: Apply horizontal flipping, scaling, translation, HSV transformation, and random erasing to $\mathbf{X}$
11: $(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}) \leftarrow \text{MixUp}(\mathbf{X}, \mathbf{Y})$ with probability $p_m = 0.15$
12: $\mathbf{F}, \mathbf{F}^{(i)}{}_{i=1}^{3} \leftarrow \text{Backbone}(\tilde{\mathbf{X}})$
13: $\mathbf{F}' \leftarrow \text{CBAM}(\mathbf{F})$
14: $\mathbf{F}'' \leftarrow \text{MSFP}(\mathbf{F}', \mathbf{F}^{(i)}{}_{i=1}^{3})$
15: $\mathbf{z} \leftarrow \text{DualPool}(\mathbf{F}'')$
16: $\hat{\mathbf{y}} \leftarrow \text{ResHead}(\mathbf{z})$
17: $\mathcal{L} \leftarrow \mathcal{L}_{\text{cls}}(\text{softmax}(\hat{\mathbf{y}}), \tilde{\mathbf{Y}}; \varepsilon, \gamma)$
18: Update θ_h using $\nabla_{\theta_h} \mathcal{L}$ with AdamW
19: **end for**
20: Update the learning rate using cosine decay
21: $A_{\text{val}} \leftarrow$ Top-1 accuracy on $\mathcal{D}_{val}$
22: **if** $A_{\text{val}} > A_{\text{best}}$ **then**
23: $A_{\text{best}} \leftarrow A_{\text{val}}$; $M^* \leftarrow$ current model
24: **end if**
25: **end for**
26: **return** M^*

initialized with value 0 in line 6. During each epoch, the training images are processed in mini-batches (lines 7–8). Each mini-batch is augmented using horizontal flipping, scaling, translation, HSV jitter, and random erasing followed by MixUp with probability p_m (lines 9–10). The augmented images are then passed through the fixed backbone to obtain the final feature map $\mathbf{F}$ and three intermediate feature maps (line 11). In line 12, CBAM applies channel and spatial attention to $\mathbf{F}$ using Eqs. (1) and (2). The resulting feature map is combined with the intermediate feature maps using MSFP according to Eq. (3) (line 13). Next, combined feature map is then converted into the feature vector $\mathbf{z}$ through average and max pooling as defined in Eq. (4) in line 14. Afterwards, residual classifier uses $\mathbf{z}$ to generate class logits $\hat{\mathbf{y}}$ following Eqs. (5)–(7) in line 15. Next, focal loss is computed using Eq. (9) and DSAC parameters θ_h are updated using AdamW (lines 16-17). After each epoch, the learning rate is reduced according to the cosine schedule and the Top-1 accuracy A_{val} is computed on the validation set (lines 19–20). If A_{val} exceeds A_{best}, A_{best} is updated and the corresponding model is saved as M^* (lines 21–22). After the final epoch, M^* is returned as the trained model (line 25).

The dataset split scales linearly with the number of samples, at $\mathcal{O}(N)$, and preprocessing with augmentation costs $\mathcal{O}(NH_mW_m)$ for images of height H_m and width W_m. Architecture setup is a one-time cost of $\mathcal{O}(|\theta_b|+|\theta_h|)$. In the forward pass, the backbone itself contributes $\mathcal{O}(B \cdot \text{FLOPs}_{\text{backbone}})$ per step. Among the DSAC modules, CBAM costs $\mathcal{O}(BCHW + BC^2/r)$ per step, MSFP costs $\mathcal{O}(3 \cdot BCHW)$, Dual-Pool costs $\mathcal{O}(BCHW)$, and the residual classifier head costs only $\mathcal{O}(BC)$, since it operates on the already-pooled feature vector rather than the full spatial map. The full training loop therefore scales as $\mathcal{O}\big(EN(CHW + C^2/r)\big)$, test-set evaluation as $\mathcal{O}(NCHW)$, and export to ONNX or CoreML INT8 is a one-time cost of $\mathcal{O}(|\theta|)$.

The overall complexity is dominated by the training loop, which scales linearly in N and E and is independent of batch size. Freezing the backbone limits backward-pass cost to the $\approx$2.68M DSAC head parameters, since $\text{FLOPs}_{\text{backbone}}$ is a fixed constant for YOLO26n-cls and gradients are not propagated through it. ImageNet-pretrained weights were used to initialize the backbone, and all DSAC head modules are trained jointly from the first epoch.

The complete pipeline was implemented using the PyTorch framework via Ultralytics and executed on an Apple M4 processor with an integrated GPU and 16 GB of RAM. We utilized the YOLO26n-cls architecture to classify six distinct maize leaf categories: Healthy, Blight, Common Rust, Gray Leaf Spot, Northern Leaf Blight (NLB), and Fall Armyworm. To improve model robustness and prevent overfitting, the training process incorporated horizontal flipping, scaling, translation, HSV color jitter, random erasing, and MixUp. YOLOv8n-cls, YOLO11n-cls, and YOLO26n-cls were fine-tuned end-to-end from ImageNet-pretrained weights for **[10]** epochs on the same $70\%/15\%/15\%$ split, whereas DSAC trains only the head on a frozen backbone. Once training concluded, the optimized model was prepared for edge deployment by exporting it to both ONNX and CoreML formats, utilizing INT8 quantization to ensure fast, low-power inference on mobile devices. The specific hyperparameters governing the training process are summarized in Table 3.

4 RESULTS

In this part, we test how well the proposed maize disease classification pipeline works by performing a series of experiments. The dataset distribution used for training, validation, and testing for all the models is summarized in Table 4.

TABLE 3: Training Hyperparameters

Parameter	Value
Number of Classes	6
Dataset Split (Train/Validation/Test)	70/15/15
Resized Image Height	H_{res}
Resized Image Width	W_{res}
Batch Size	32
Epochs	10
Learning Rate	0.001
Optimizer	AdamW
Channel-Attention Bottleneck Ratio (r)	16
Label Smoothing (ε)	0.1
Focal Term (γ)	1.5
MixUp Interpolation Coefficient (p_m)	0.15

TABLE 4: Dataset Distribution for Maize Disease Classification

Class	Train	Validation	Test	Total
Blight	802	172	172	1146
Northern Leaf Blight	700	150	150	1000
Common Rust	914	196	196	1306
Fall Armyworm	153	33	33	219
Gray Leaf Spot	401	86	87	574
Healthy	1162	249	250	1661

The analysis encompasses training behavior, individual model performance, a comparison with various baseline architectures, and a qualitative assessment of predictions.To keep things fair, all experiments are carried out on the same test split under same hardware and software settings. Additional details about the setup and dataset are given in Table 3 and Table 4.

4.1 Experimental Setting

4.1.1 Dataset Split and Evaluation Metric

The dataset consists of 5,906 images distributed across six classes, including healthy maize leaves. The dataset was divided into training, validation, and test sets using a 70:15:15 split. This resulted in 4,132 images for training, 886 images for validation, and 888 images for testing. The same data split was used for the models evaluated in this study. Classification performance was evaluated using Top-1 accuracy, which measures the proportion of test images for which the predicted class corresponds to the ground truth class.

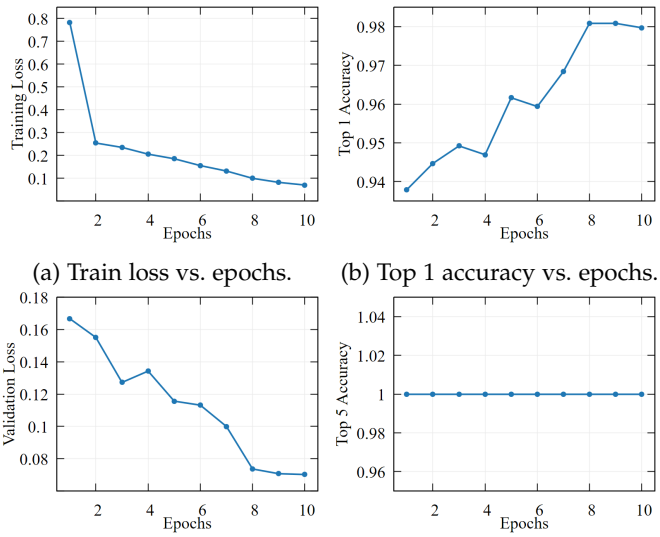
4.1.2 Hyperparameter Setting

For the proposed model, the input image resolution was resized to $H_{res} \times W_{res}$ pixels. The model was optimized using the AdamW optimizer with an initial learning rate of $5{\times}10^{-4}$ and a weight decay of $5{\times}10^{-4}$. A warmup period of five epochs was used at the beginning of training, followed by cosine learning rate decay with a final learning rate factor of 0.001. Label smoothing was employed to reduce overconfident predictions, while a dropout value of 0.3 was used in the training configuration.

Data augmentation was applied during training to improve the robustness of the model to variations in leaf appearance. The augmentation pipeline included horizontal flipping, scaling, translation, HSV based changes in hue, saturation and brightness, image erasing, and MixUp. The validation and test images were evaluated without these training time augmentations.

4.2 Training Convergence

The model shows stable convergence during training, as shown in Fig. 3. The training loss steadily drops from about 0.83 in the first epoch to almost 0.06 by epoch 10. This means that the model starts to learn important features early on in training.



(a) Train loss vs. epochs. (b) Top 1 accuracy vs. epochs.

(c) Validation loss vs. epochs. (d) Top 5 accuracy vs. epochs.

Fig. 3: Analysis of loss and accuracy of the proposed model.

It is worth noting that with K = 6 disease classes, Top-5 accuracy is a trivially near-perfect metric by definition — a correct prediction need only fall within five of six possible classes — and is therefore not a discriminating indicator of model quality in this setting. Accordingly, primary evaluation emphasis is placed on Top-1 accuracy and macro-average F1, which are more meaningful under small K.

The validation loss follows same general pattern. It shows a slight increase during the initial epochs—probably because the feature space is being explored—before gradually decreasing and stabilizing around 0.07. There is no major gap between training and validation loss, which indicates that the model is able to generalize well without clear indications of overfitting, even with a comparatively small dataset.

The Top-1 accuracy on the validation set improves steadily across epochs and over time reaches approximately 98.09%. A minor transient dip in accuracy is observed in the middle epochs, consistent with typical stochastic gradient noise during head training, before recovering and stabilizing. The Top-5 accuracy remains near 100% throughout training, confirming that the ground-truth class is almost always among the model's top predictions.

4.3 Classification Performance

The confusion matrix in Fig. 4 shows all of the model's predictions for all classes in great detail. Most of the values are along the diagonal, which means that the classifications are correct.

The *Northern Leaf Blight*, *Common Rust* and *Healthy Maize* classes achieve a perfect recall of 1.00, with no misclassifications observed across the entire test set. The classes

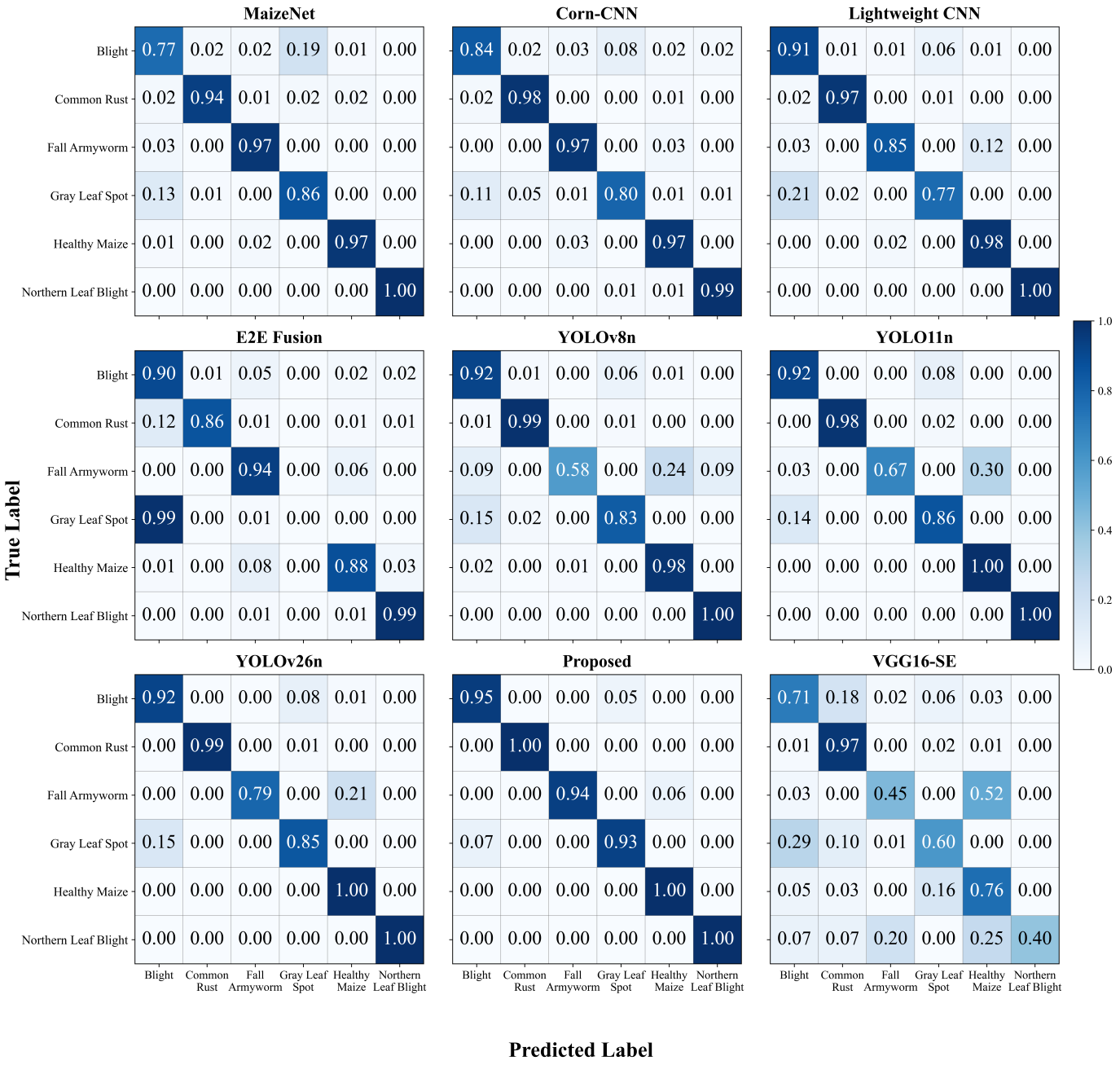


Fig. 4: Normalised confusion matrices on the held-out test set for all included models.

likewise show near-perfect performance, with recall values of 1.00 and 0.99, respectively, and no significant inter-class confusion. The *Fall Armyworm* class achieves a recall of 0.94, representing an improvement over YOLOv8n-cls, YOLO11n-cls, and YOLO26n-cls which achieve 0.58, 0.67, and 0.79 respectively for this class.

The most challenging classes are *Blight* and *Gray Leaf Spot*, which exhibit moderate inter-class confusion owing to overlapping visual characteristics such as lesion shape, colour, and texture. Specifically, Blight samples are occasionally misclassified as Gray Leaf Spot (0.05 confusion rate) and Gray Leaf Spot as Blight (0.07 confusion rate). This behaviour is consistent across all evaluated models and reflects an inherent difficulty in visually separating these two disease categories, rather than a model-specific limitation. Despite this, the proposed model reduces the confusion rate for both classes compared to the baseline YOLO variants.

4.4 Error Analysis

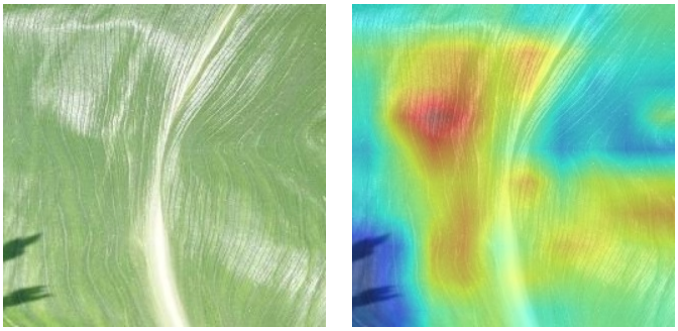
A systematic analysis of classification errors reveals that nearly all misclassifications occur between visually similar disease pairs. The main source of misclassifications comes from the confusion between Blight and Gray Leaf Spot. Under different lighting and imaging conditions, these classes show similar long lesion shapes and colour patterns that overlap. There is also some confusion in the Fall Armyworm

class because feeding damage can sometimes look like mechanical environmental leaf injuries in other groups.

Interestingly, these trends appear across all the models we evaluated. Which suggests that the difficulty is not primarily due to limitations of a specific architecture, but rather due to the visual similarity between certain disease patterns in the dataset. Techniques such as multispectral imaging or higher-resolution image capture could potentially further reduce inter-class confusion in future work.

4.5 Grad-CAM Visualization Analysis

Fig.5 presents a qualitative comparison between a raw maize leaf image and its corresponding GradCAM activation map. The heatmap reveals that the model's attention concentrates on the discoloured lesion regions and irregular boundary patterns, rather than on background vegetation or soil. This spatial alignment between model activations and symptom regions offers interpretable evidence that the classification decisions are influenced by disease relevant features rather than background elements [19].



(a) Original Image (b) GradCAM Heatmap

Fig. 5: Comparison of Original Image and GradCAM Heatmap

4.6 Quantitative Results

Based on the evaluation of the held-out test set, the proposed model achieves the following performance :

- Top-1 Accuracy: approximately 98.09%
- Macro-average F1 Score: 97.3%
- Macro Precision / Recall: 97.6% / 96.9%
- CPU Inference Latency: 8.4 ms (119 FPS)
- GPU Inference Latency: 4.7 ms (215 FPS)
- Model Parameters: 1.534 M
- ONNX Model: Top-1 Accuracy: approximately 97.61%, CPU Inference Latency: 8.6 ms (116 FPS)
- CoreML INT8 Model: Top-1 Accuracy: approximately 97.61%, Inference Latency: 2.4 ms (417 FPS)

These results confirm that the proposed model maintains high predictive reliability in all six classes simultaneously while meeting the strict latency and resource constraints of smartphone deployment.

4.7 Model Comparison

To evaluate the proposed approach, a comparative study is conducted across nine architectures on the same test split under identical conditions, covering accuracy, confusion structure, per-class F1, precision-recall, and inference efficiency. The nine models are: MaizeNet [2]; Corn-CNN; LW-CNN; E2E Fusion; YOLOv8n-cls [11]; YOLO11n-cls; YOLOv26n-cls [20]; VGG16-SE; and the Proposed Model. E2E Fusion was trained on RGB images only, with the sensor branch removed

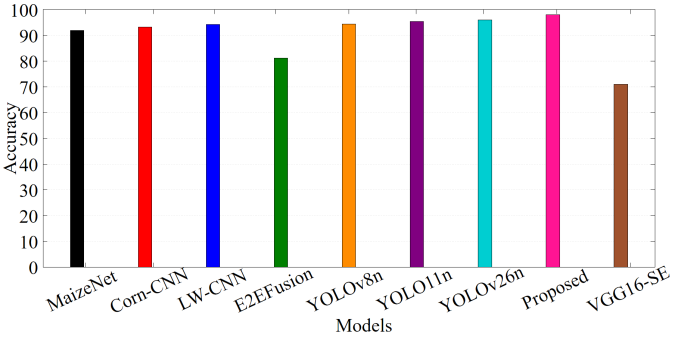


Fig. 6: Top-1 classification accuracy for all nine evaluated models on the held-out test set.

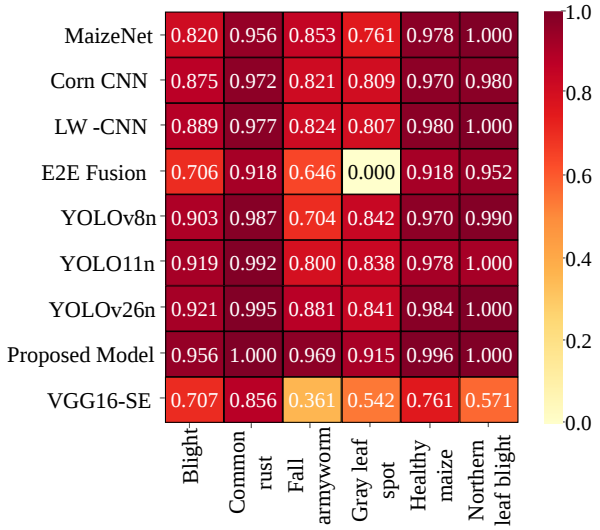


Fig. 7: F1 score heatmap for each class across all nine evaluated models.

4.7.1 Overall Accuracy Comparison

Fig.6 presents the Top-1 test accuracy. The proposed model achieves the highest accuracy at **98.09**%. Among YOLO variants, YOLOv26n-cls achieves 95.95%, YOLO11n-cls 95.38%, and YOLOv8n-cls 94.37%. Classical baselines range from 81.19% (E2E Fusion) to 94.26% (LW-CNN), while SE-block variant perform poorly at 70.95% (VGG16-SE).

4.7.2 Per-Class F1 Score Analysis

Proposed model achieves the highest F1 in all six classes. The macro-average F1 of 97.3% surpasses YOLOv8n-cls (89.9%), YOLO11n-cls (92.1%) and YOLOv26n-cls (93.7%). Key gains are in *Fall Armyworm* and *Gray Leaf Spot*. Full per-class scores are shown in Fig. 7.

TABLE 5: Comprehensive Performance Comparison of Models

Model	Top-1 (%)	Macro F1 (%)	Precision (%)	Recall (%)	Parameters(M)	Latency CPU/GPU (ms)
MaizeNet	91.89	89.5	88.0	91.8	1.435	7.20 / 1.29
Corn-CNN	93.24	90.5	89.2	92.4	0.830	15.33 / 2.14
LW-CNN	94.26	91.3	91.3	91.3	4.196	25.38 / 0.86
E2E Fusion	81.19	69.0	65.7	76.0	5.117	137.56 / 17.33
YOLOv8n-cls	94.37	89.9	92.9	88.2	1.443	7.38 / 4.56
YOLO11n-cls	95.38	92.1	94.9	90.5	1.534	9.94 / 4.85
YOLO26n-cls	95.95	93.7	95.4	92.4	1.534	9.43 / 5.12
VGG16-SE	70.95	63.3	67.1	64.9	15.093	153.87 / 143.59
Proposed Model	**98.09**	**97.3**	**97.6**	**96.9**	**4.214**	**8.39 / 4.65**

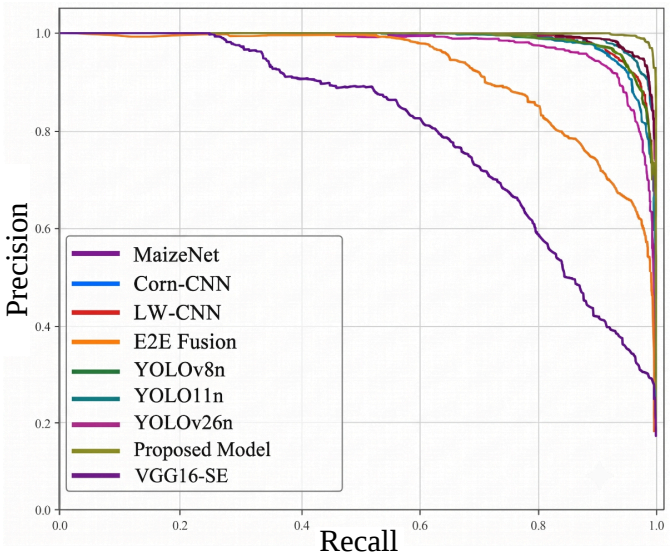


Fig. 8: Macro-averaged Precision–Recall curves for all nine evaluated models.

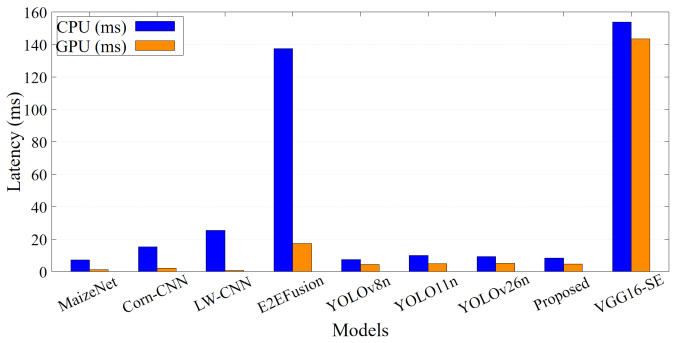


Fig. 9: CPU and GPU inference latency (batch size = 32) across all nine evaluated models.

4.7.3 Precision-Recall Analysis

Proposed model achieves the highest macro PR-AUC of 0.9948, followed by YOLOv26n-cls (0.9812) and YOLO11n-cls (0.9791). YOLOv8n-cls achieves 0.9536, MaizeNet achieves 0.9492, Corn-CNN achieves 0.9667 and E2E Fusion achieves 0.8295. Proposed model maintains high precision at recall > 0.9, confirming stable confidence calibration across all classes. Figure 8 shows the precision–recall curves for all the models that were evaluated. The proposed model keeps precision close to one over a wide range of recall values. Its curve stays near the right corner of the plot, which is a good sign. YOLO11n-cls and YOLO26n-cls follow patterns but their precision drops more clearly as recall increases. E2E Fusion also sees a drop in precision especially after recall reaches about 0.6. VGG16-SE has a decline starting at much lower recall values. In general the proposed model offers a balance, between precision and recall especially when recall is high.

4.7.4 Inference Efficiency

Among YOLO variants, YOLOv8n-cls achieves 7.4 ms CPU and 4.6 ms GPU latency, marginally faster than Proposed Model (8.4 ms CPU and 4.7 ms GPU). E2E Fusion requires 137.6 ms CPU, over 13× slower. VGG16-SE (153.9 ms CPU) precludes real-time edge deployment entirely. Despite nearly identical latency to YOLOv8n-cls, Proposed model delivers $+3.72\%$ higher accuracy and $+7.4\%$ better macro F1. Full results are in Table 5.

Overall, Proposed Model achieves a favorable balance between performance and computational cost, validating its effectiveness for the classification of maize disease in real-time.

4.8 Qualitative Analysis

Fig. 10 displays representative sample predictions from the validation set in all six disease categories. The model assigns high confidence scores (frequently ≥ 0.99) to correct predictions, reflecting sharp class probability distributions. The model works well in a range of real-world situations, such as changes in the orientation of leaves, partial obstacles, lighting, and background elements such as soil and vegetation nearby.

4.9 Mobile Application

To validate the proposed framework in the field, we built an offline Android application using Flutter, running the exported ONNX model entirely on-device via the *flutter_onnxruntime* plugin (ONNX Runtime 1.22.0, opset 19). The app has a simple 3-screen flow (Fig. 11): the *Home Screen* allows the user to capture an image or select one from the gallery; the *Detection Screen* performs on-device inference with a loading indicator; and the *Result Screen* displays the leaf image, the predicted disease label, and a colour-coded confidence bar (green for healthy, red for diseased), with confidence displayed numerically and visually. Preprocessing is same as the training pipeline — where each image is resized to $H_{res} \times W_{res}$ and normalized channel-wise using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma =$

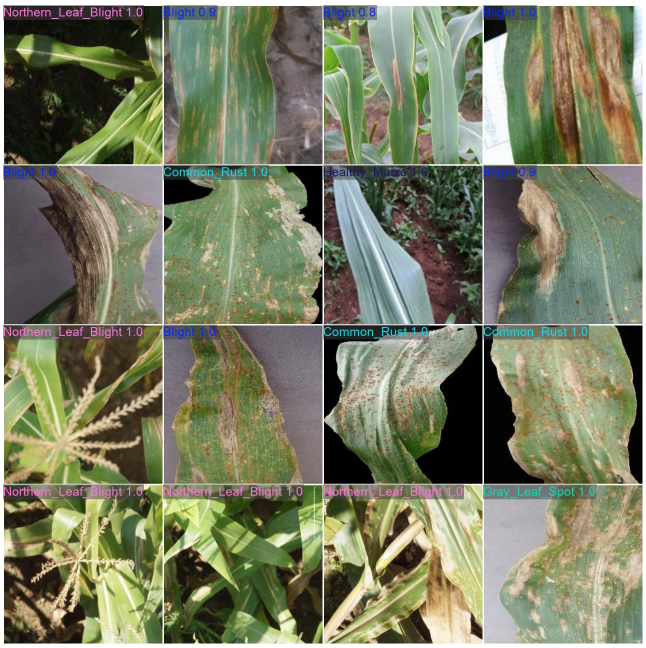


Fig. 10: Sample predictions showing model outputs with corresponding confidence scores.

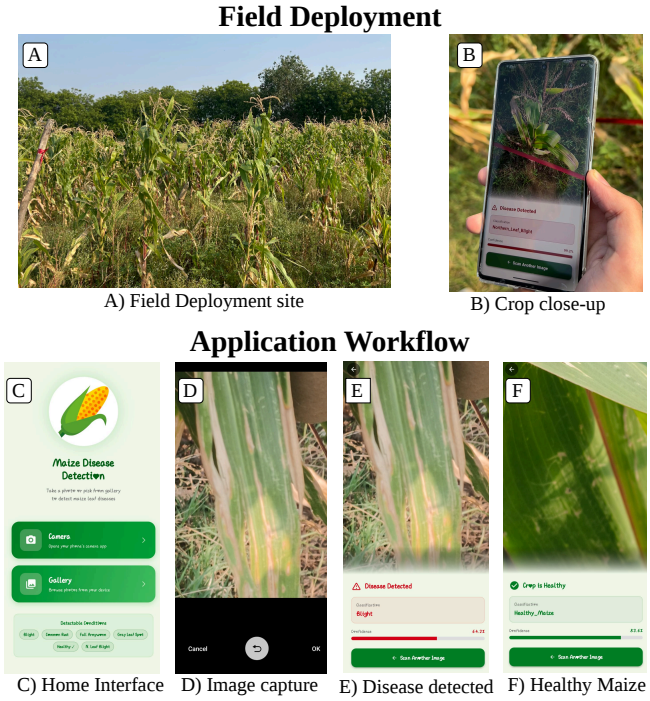


Fig. 11: Field deployment and application workflow

$[0.229, 0.224, 0.225]$), and converted to a BCHW `Float32` tensor — and the model is bundled within the app, so no internet connection is needed at any stage.

The application was evaluated on a Google Pixel 7 Pro (Android 15), equipped with a 50-MP rear wide camera (1/1.31-inch sensor, 1.2-µm pixel size, f/1.85 aperture, and 82° field of view). Image acquisition was performed using the default rear-facing camera at 1× zoom, without digital zoom. The captured images had a resolution of 900×1600 pixels (1.44 MP) and were subsequently resized to $H_{res} \times W_{res}$ pixels. The app achieves an average end-to-end inference latency of 12–15 ms per image on-device, including preprocessing, with a cold-start model-loading time of ≈800 ms (cached thereafter). The packaged `arm64-v8a` APK is around 45 MB, suitable for standard app-store distribution, and does not require internet permission since all processing and image data stay on-device.

TABLE 6: Deployment Benchmark on Mobile Formats

Format	Accuracy (%)	Latency (ms)
ONNX	97.61	8.6
CoreML INT8	97.61	2.4

5 Conclusion

Running disease detection entirely on a farmer's phone means no cloud, no network dependency, and no room for a heavyweight model - yet the accuracy still has to be trustworthy enough to act on. This paper shows that the classification head, not just the backbone, is where that trade-off is won. The proposed DSAC head attaches five lightweight components - CBAM attention, multi-scale feature fusion, dual-pooling, a residual classifier, and focal-aware label smoothing - to a frozen YOLO26n-cls backbone, raising Top-1 accuracy from 95.95% to 98.09% and macro-F1 from 93.7% to 97.3%, at a CPU latency cost of under a millisecond.

The gains are not uniform. Blight and Gray Leaf Spot remain hard to tell apart, a confusion that shows up across all nine architectures we tested and looks like a property of the dataset rather than a flaw in any one model. Fall Armyworm tells the opposite story: recall jumps to 0.94 from 0.58-0.79 across the YOLO baselines, and Grad-CAM confirms that the model is reading lesion tissue rather than background leaf. We carried the model through to a working Flutter app - 45 MB, fully offline, 12-15 ms per image on a Pixel 7 Pro, and 2.4 ms via CoreML INT8 on Apple Macbook Air M4, with no retraining needed for either export. Closing the remaining Blight-Gray Leaf Spot gap likely needs better data, higher resolution or multispectral capture, more than further architectural changes, and adding severity estimation on top of classification is the natural next step toward a tool a farmer could actually use in the field.

References

[1] Reenu Susan Joseph, Nisha C.A., and Shandry K. K. Plant disease detection: A survey on recent deep learning technologies. In *2022 Second International Conference on Next Generation Intelligent Systems (ICNGIS)*, pages 1–6, 2022.

[2] Mutia Fadhilla, Des Suryani, Ause Labella, and Hendra Gunawan. Corn leaf diseases recognition based on convolutional neural network. *IT Journal Research and Development*, 8:14–21, 08 2023.

[3] Pichayoot Ouppaphan. Corn disease identification from leaf images using convolutional neural networks. In *2017 21st International Computer Science and Engineering Conference (ICSEC)*, pages 1–5, 2017.

[4] Mohammad Yasin Arefin, Rawhatur Rabbi, Iffat Fahmida Turna, Zahra Zannat, and Dewan Ziaul Karim. Lightweight custom cnns take on corn leaf disease: Deep learning for plant pathology. In *2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, pages 1–6, 2023.

[5] Hassan Amin, Ashraf Darwish, Aboul Ella Hassanien, and Mona Soliman. End-to-end deep learning model for corn leaf disease classification. *IEEE Access*, 10:31103–31115, 2022.

[6] Abu Sayem Md. Siam, Abir Hossain, Rifat Bin Hossain, and Md. Musfiqur Rahman. Se-vgg16 maizenet: Maize disease classification using deep learning and squeeze and excitation attention networks. In *2024 International Conference on Emerging Smart Computing and Informatics (ESCI)*, pages 1–6, 2024.

[7] Fikadu Weloday and Jianmei Su. Lwmscnn-se: A lightweight multi-scale network for efficient maize disease classification on edge devices. *2026 International Conference on Communication Networks and Machine Learning (CNML)*, pages 807–814, 2026.

[8] Fikadu Weloday and Jianmei Su. Lwmscnn-se: A lightweight multi-scale network for efficient maize disease classification on edge devices, 2026.

[9] Peruvazhuthi Srinivasan and Priyanka Kokil. A residual attention framework for corn leaf disease diagnosis under resource constraints. In *2026 IEEE Guwahati Subsection Conference (GCON)*, pages 1–5, 2026.

[10] Priyanto Hidayatullah, Nurjannah Syakrani, Muhammad Rizqi Sholahuddin, Trisna Gelar, and Refdinal Tubagus. Yolov8 to yolo11: A comprehensive architecture in-depth comparative review, 2025.

[11] Priyanto Hidayatullah and Refdinal Tubagus. Yolo26: A comprehensive architecture overview and key improvements, 2026.

[12] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. Cbam: Convolutional block attention module, 2018.

[13] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 936–944, 2017.

[14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

[15] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023.

[16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[17] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016.

[18] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2999–3007, 2017.

[19] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[20] Ranjan Sapkota, Rahul Harsha Cheppally, Ajay Sharda, and Manoj Karkee. Yolo26: Key architectural enhancements and performance benchmarking for real-time object detection, 2026.

[21] Max Sponner, Bernd Waschneck, and Akash Kumar. Ai-driven performance modeling for ai inference workloads. *Electronics*, 11(15), 2022.

[22] Yong Zeng, Jie Xu, and Rui Zhang. Energy minimization for wireless communication with rotary-wing uav. *IEEE Transactions on Wireless Communications*, PP, 04 2018.

[23] Xinda Liu, Weiqing Min, Shuhuan Mei, Lili Wang, and Shuqiang Jiang. Plant disease recognition: A large-scale benchmark dataset and a visual region and loss reweighting approach. *IEEE Transactions on Image Processing*, 30:2003–2015, 2021.

[24] Rohit Rama Subramaniam, Om Anand, Shankar Thangavelu, and S. Ravi. Improved energy efficient localization of uavs in farmlands using hybrid metaheuristic optimization algorithms. *Energy Conversion and Management: X*, 30:101666, 2026.

[25] Gonireddy Rishitha, Venkata Sudhakar Chowdam, Harshavardhan Reddy Gunnamreddy, Ashwitha Boddu, and Manopriya Halaharvi. Plant leaf disease detection by using mobilenet and inception. In *2025 International Conference on Computing and Communication Technologies (ICCCT)*, pages 1–6, 2025.

[26] U. Archana, Amanulla Khan, Appani Sudarshanam, C. Sathya, Ashok Kumar Koshariya, and R. Krishnamoorthy. Plant disease detection using resnet. In *2023 International Conference on Inventive Computation Technologies (ICICT)*, pages 614–618, 2023.

[27] T Devi., K. Alice, and N. Deepa. Plant disease detection using enhanced efficientnet architecture in comparison with densenet to analyze the severity in leaves with performance measures. In *2022 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI)*, volume 01, pages 1–5, 2022.

[28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[29] Food and Agriculture Organization of the United Nations. Faostat: Food and agricultural data. http://www.fao.org/faostat/, 2019.

[30] Yang Wang, Dazheng Deng, Leibo Liu, Shaojun Wei, and Shouyi Yin. Pl-npu: An energy-efficient edge-device dnn training processor with posit-based logarithm-domain computing. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 69(10):4042–4055, 2022.

[31] Jiasi Chen and Xukan Ran. Deep learning with edge computing: A review. *Proceedings of the IEEE*, 107(8):1655–1674, 2019.

[32] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *CoRR*, abs/1905.11946, 2019.

[33] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *CoRR*, abs/2010.11929, 2020.

[34] Apple Inc. Core ml documentation. https://developer.apple.com/documentation/coreml, 2023. Accessed: 2026-05-04.

[35] ONNX Community. Onnx: Open neural network exchange. https://onnx.ai/, 2024. Accessed: 2026-05-04.

[36] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.

[37] Hongyi Zhang, Moustapha Cissé, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. *CoRR*, abs/1710.09412, 2017.

[38] Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. *CoRR*, abs/1711.05101, 2017.